

CHAPTER 10

Using Graph Analytics for Big Data

While we have discussed how the various techniques and technologies for big data appliances can increase performance (in terms of scalability for computation speed or for data volumes), we have also seen that the ways that certain applications are mapped to a typical big data stack might have limitations in scalability due to memory access latency or network bandwidth. Yet the promise of big data must go beyond increased scalability for known problems.

Big data analytics systems should enable a platform that can support different analytics techniques that can be adapted in ways that help solve a variety of challenging problems. This suggests that these systems are high performance, elastic distributed data environments that enable the use of creative algorithms to exploit variant modes of data management in ways that differ from the traditional batch-oriented approach of traditional approaches to data warehousing.

In this chapter, we look at graph analytics, which is an analytics alternative that uses an abstraction called a **graph model**. The simplicity of this model allows for rapidly absorbing and connecting large volumes of data from many sources in ways that finesse limitations of the source structures (or lack thereof, of course). Graph analytics is an alternative to the traditional data warehouse model as a framework for absorbing both structured and unstructured data from various sources to enable analysts to probe the data in an undirected manner.

As we will see, the graph model allows you to tightly couple the meaning of entity relationships as part of the representation of the relationship. This effectively embeds the semantics of relationships among different entities within the structure, providing an ability to both invoke traditional-style queries (to answer typical “search” queries modeled after known patterns) and enable more sophisticated undirected analyses. These undirected “discovery” analyses, include inferencing, identification of interesting patterns, and application of deduction, all using an iterative approach that analysts can use to discover actionable knowledge that was previously unknown. This allows

Big Data Analytics. DOI: <http://dx.doi.org/10.1016/B978-0-12-417319-4.00010-7>

© 2013 Elsevier Inc. All rights reserved.

the analysts to rapidly seek out emerging patterns and enable real-time knowledge-driven decision making in the context of how these newly discovered patterns impact the corporate business value drivers.

10.1 WHAT IS GRAPH ANALYTICS?

It is worth delving somewhat into the graph model and the methods used for managing and manipulating graphs:

- What constitutes graph analytics?
- Types of problems that are suited to graph analytics.
- Types of questions that are addressed using graph analytics.
- Types of graphs that are commonly encountered.
- The degree of prevalence within big data analytics problems.

This motivates an understanding of its utility and flexibility for discovery-style analysis in relation to specific types of business problems, how common those types of problems are, and why they are nicely abstracted to the graph model. In addition, we will discuss the challenges of attempting to execute graph analytics on conventional hardware and consider aspects of specialty platforms that can help in achieving the right level of scalability and performance.

10.2 THE SIMPLICITY OF THE GRAPH MODEL

Graph analytics is based on a model of representing individual entities and numerous kinds of relationships that connect those entities. More precisely, it employs the graph abstraction for representing connectivity, consisting of a collection of *vertices* (which are also referred to as *nodes* or *points*) that represent the modeled entities, connected by *edges* (which are also referred to as *links*, *connections*, or *relationships*) that capture the way that two entities are related.

Some simple examples are shown in [Figure 10.1](#).

The flexibility of the model is based on its simplicity. A simple unlabeled undirected graph, in which the edges between vertices neither reflect the nature of the relationship nor indicate their direction, has limited utility. But, as we see in the examples in [Figure 10.1](#), adding context by labeling the vertices and edges enhances the meanings embedded within the graph, and by extension, the entire representation of a network.

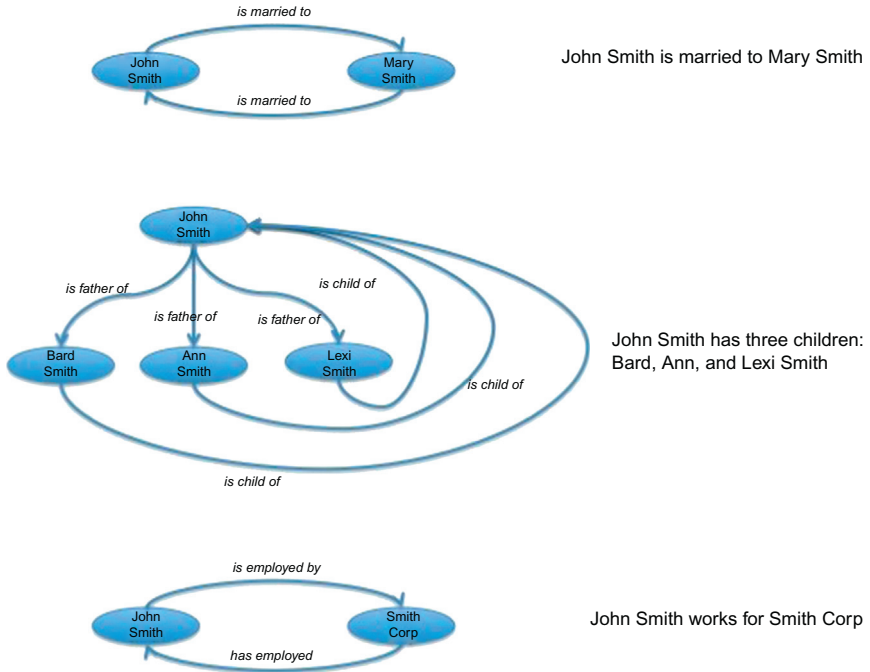


Figure 10.1 Examples of relationships represented as labeled directed graphs.

Among other enhancements, these can enrich the meaning of the nodes and edges represented in the graph model:

- Vertices can be labeled to indicate the types of entities that are related.
- Edges can be labeled with the nature of the relationship.
- Edges can be directed to indicate the “flow” of the relationship.
- Weights can be added to the relationships represented by the edges.
- Additional properties can be attributed to both edges and vertices.
- Multiple edges can reflect multiple relationships between pairs of vertices.

10.3 REPRESENTATION AS TRIPLES

In essence, these enhancements help in building a semantic graph—a directed graph that can be represented using a *triple* format consisting of a *subject* (the source point of the relationship), an *object* (the target), and a *predicate* (that models the type of the relationship).

The same examples in [Figure 10.1](#) could be modeled as a set of triples, as shown in [Table 10.1](#).

A collection of these triples is called a semantic database, and this kind of database can capture additional properties of each triple relationship as attributes of the triple. Almost any type of entity and relationship can be represented in a graph model, which means two key things: the process of adding new entities and relationships is not impeded when new datasets are to be included, with new types of entities and connections to be incorporated, and the model is particularly suited to types of discovery analytics that seek out new patterns embedded within the graph that are of critical business interest.

10.4 GRAPHS AND NETWORK ORGANIZATION

The concept of the social network has been around for many years, and in recent times has been materialized as communities in which individual entities create online personas and connect and interact with others within the community. Yet the idea of the social network extends beyond specific online implementations, and encompasses a wide variety of example environments that directly map to the graph model. That being said, one of the benefits of the graph model is the ability to detect patterns or organization that are inherent within the represented network, such as:

- **Embedded micronetworks:** Looking for small collections of entities that form embedded “microcommunities.” Some examples include determining the originating sources for a hot new purchasing trend,

Table 10.1 Triples Derived from the Example in Figure 10.1		
Subject	Predicate	Object
John Smith	Is married to	Mary Smith
Mary Smith	Is married to	John Smith
John Smith	Is father of	Brad Smith
John Smith	Is father of	Ann Smith
John Smith	Is father of	Lexi Smith
Brad Smith	Is child of	John Smith
Ann Smith	Is child of	John Smith
Lexi Smith	Is child of	John Smith
John Smith	Is employed by	Smith Corp
Smith Corp	Has employed	John Smith

identifying a terrorist cell based on patterns of communication across a broad swath of call detail records, or sets of individuals within a particular tiny geographic region with similar political views.

- **Communication models:** Modeling communication across a community triggered by a specific event, such as monitoring the “buzz” across a social media channel associated with the rumored release of a new product, evaluating best methods for communicating news releases, or correlation between travel delays and increased mobile telephony activity.
- **Collaborative communities:** Isolating groups of individuals that share similar interests, such as groups of health care professionals working in the same area of specialty, purchasers with similar product tastes, or individuals with a precise set of employment skills.
- **Influence modeling:** Looking for entities holding influential positions within a network for intermittent periods of time, such as computer nodes that have been hijacked and put to work as proxies for distributed denial of service attacks or for emerging cybersecurity threats, or individuals that are recognized as authorities within a particular area.
- **Distance modeling:** Analyzing the distances between sets of entities, such as looking for strong correlations between occurrences of sets of statistically improbable phrases among large sets of search engines queries, or the amount of effort necessary to propagate a message among a set of different communities.

Each of these example applications is a discovery analysis that looks for patterns that are not known in advance. As a result, these are less suited to pattern searches from relational database systems, such as a data warehouse or data mart, and are better suited to a more dynamic representation like the graph model.

10.5 CHOOSING GRAPH ANALYTICS

Deciding the appropriateness of an analytics application to a graph analytics solution instead of the other big data alternatives can be based on these characteristics and factors of business problems:

- **Connectivity:** The solution to the business problem requires the analysis of relationships and connectivity between a variety of different types of entities.

- **Undirected discovery:** Solving the business problem involves iterative undirected analysis to seek out as-of-yet unidentified patterns.
- **Absence of structure:** Multiple datasets to be subjected to the analysis are provided without any inherent imposed structure.
- **Flexible semantics:** The business problem exhibits dependence on contextual semantics that can be attributed to the connections and corresponding relationships.
- **Extensibility:** Because additional data can add to the knowledge embedded within the graph, there is a need for the ability to quickly add in new data sources or streaming data as needed for further interactive analysis.
- **Knowledge is embedded in the network:** Solving the business problem involves the ability to exploit critical features of the embedded relationships that can be inferred from the provided data.
- **Ad hoc nature of the analysis:** There is a need to run ad hoc queries to follow lines of reasoning.
- **Predictable interactive performance:** The ad hoc nature of the analysis creates a need for high performance because discovery in big data is a collaborative man/machine undertaking, and predictability is critical when the results are used for operational decision making.

10.6 GRAPH ANALYTICS USE CASES

Having reviewed the business problems that are suited to using a solution based on a graph analytics model, we can examine some use cases for graph analytics. In these scenarios, we look at the business problem, why the traditional approach may not be optimal, and then discuss how the graph analytics approach is best suited to solving the business challenges.

Some example business applications that are aligned with these criteria include:

- **Health care quality analytics,** in which patient health encounter histories, diagnoses, procedures, treatment approaches, and results of clinical trials contribute to the ability to analyze the comparative effectiveness of different health care options. The process requires the absorption of many different collections of medical histories, clinical records, laboratory results, and prescription records from many different sources and systems. In turn, the application must

guide health care providers in a timely and efficient manner by enabling the rapid assessment of therapies used for other patients with similar characteristics (such as age, clinical history, and associated risk factors) that have the most positive outcomes. This analysis creates a graph representing the corresponding relationships and may also combine health histories with additional data sources that can be easily integrated into the existing graph model.

- **Concept-based correlations** that seek to organize large bodies of knowledge. Some examples include looking for contextual relationships between scientific health care research and particular types of pharmaceuticals, investigative journalists seeking connections among individuals referred to in a variety of news sources, fraud analysts evaluating financial irregularities across multiple-related organizations, or even correlations between corporate behavior and increased health risks. Each of these examples involve the absorption of many content artifacts from multiple varied data sources, the need to combine isolated pieces of information extracted from among the corpus of documents, and a discovery activity looking for correlations that are currently hidden or unknown.
- **Cybersecurity:** The numbers of cyber attacks are increasing, and their sophistication is expanding way beyond distributed denial of service (DDoS), which is more frequently being used as a premise and distraction while more insidious attacks seek to traverse corporate firewalls, extract critical business information, or incrementally drain individual financial accounts while operating completely under the radar. Monitoring for cybersecurity events is a process in which a wide variety of massive streaming datasets (such as network logs, NetFlow, DNS, and IDS data) need to be rapidly captured and integrated into a model that allows for both identification of known attack patterns as well as the discovery of new patterns that emerge as the attacks become more elaborate. A graph analytics approach can address the challenge by capturing and loading the entire volume and breadth of the available datasets coupled with an evolving model that can quickly log connections and relationships that is used to identify new patterns of attack. This graph-based approach allows analysts to rapidly get results from many ad hoc queries requested from a model managing massive amounts of data representing thousands of interconnected entities. This approach enables the analysts to quickly identify potential cyber-threats in minutes, or even seconds so that defensive actions can be taken quickly.

10.7 GRAPH ANALYTICS ALGORITHMS AND SOLUTION APPROACHES

The graph model is inherently suited to enable a broad range of analyses that are generally unavailable to users of a standard data warehouse framework. As suggested by these examples, instead of just providing reports or enabling online analytical processing (OLAP) systems, graph analytics applications employ algorithms that traverse or analyze graphs to detect and potentially identify interesting patterns that are sentinels for business opportunities for increasing revenue, identifying security risks, detecting fraud, waste, or abuse, financial trading signals, or even looking for optimal individualized health care treatments. Some of the types of analytics algorithmic approaches include:

- **Community and network analysis**, in which the graph structures are traversed in search of groups of entities connected in particularly “close” ways. One example is a collection of entities that are completely connected (i.e., each member of the set is connected to all other members of the set).
- **Path analysis**, which analyze the shapes and distances of the different paths that connect entities within the graph.
- **Clustering**, which examines the properties of the vertices and edges to identify characteristics of entities that can be used to group them together.
- **Pattern detection and pattern analysis**, or methods for identifying anomalous or unexpected patterns requiring further investigation.
- **Probabilistic graphical models** such as Bayesian networks or Markov networks for various application such as medical diagnosis, protein structure prediction, speech recognition, or assessment of default risk for credit applications.
- **Graph metrics** that are applied to measurements associated with the network itself, including the degree of the vertices (i.e., the number of edges in and out of the vertex), or centrality and distance (including the degree to which particular vertices are “centrally located” in the graph, or how close vertices are to each other based on the length of the paths between them).

These graph analytic algorithms can yield interesting patterns that might go undetected in a data warehouse model, and these patterns themselves can become the templates or models for new searches. In other

words, the graph analytics approach can satisfy both the discovery and the use of patterns typically used for analysis and reporting.

10.8 TECHNICAL COMPLEXITY OF ANALYZING GRAPHS

As more use cases similar to the ones shared in this chapter emerge that are expected to consume and analyze massive datasets, the question of performance gradually stands out as one of the potential barriers to success. To what extent do big data platforms effectively satisfy the demand for high performance? Answering that question requires a better understanding of some of the complexities of graph analytics problems and topologies.

There are some characteristics of graphs that inhibit the ability of typical computing platforms to provide the rapid responses or satisfy the need for scalability, especially as data volumes continue to explode. Some of the factors that might introduce performance penalties that are not easily addressed on standard hardware architectures include:

- **Unpredictability of graph memory accesses:** The types of discovery analyses in graph analytics often require the simultaneous traversal of multiple paths within a network to find the interesting patterns for further review or optimal solutions to a problem. The graph model is represented using data structures that incorporate the links between the represented entities, and this differs substantially from a traditional database model. In a parallel environment, many traversals can be triggered at the same time, but each graph traversal is inherently dependent on the ability to follow the links from the subject to the target in the representation. Unlike queries to structured databases, the memory access patterns are not predictable, limiting the ability to reduce data access latency by efficiently streaming pre-fetched data through the different levels of the memory hierarchy, as briefly discussed in Chapter 6. Much execution time is spent accessing memory, while the processors wait for the memory accesses to complete.
- **Graph growth models:** As more information is introduced into an environment, real-world networks grow in an interesting way. The larger the graph becomes, the more it exhibits what is called “preferential connectivity”—newly introduced entities are more likely to

connect to already existing ones, and existing nodes with a high degree of connectivity are more likely to continue to be “popular” in that they will continue to attract new connections. This means that graphs continue to grow, but apparently that growth does not scale equally across the data structure.

- **Dynamic interactions with graphs:** As with any big data application, graphs to be analyzed are populated from a wide variety of data sources of large or massive data volumes, streamed at varying rates. But while the graph must constantly absorb many rapidly changing data streams and process the incorporation of connections and relationships into the persistent representation of the graph, the environment must also satisfy the need to rapidly respond to many simultaneous discovery analyses. Therefore, a high-performance graph analytics solution must accommodate the dynamic nature without allowing for any substantial degradation in analytics performance.
- **Complexity of graph partitioning:** Graphs tend to cluster among centers of high connectivity, as many networks naturally have “hubs” consisting of a small group of nodes with many connections. The benefit of having hubs is that in general it shortens the distances among collections of nodes in the graph. The hubs often showcase entities that may be of particular interest because of their perceived centrality. But while one of the benefits of big data platforms is the expectation of distribution of data and computation, the existence of hubs make it difficult to partition the graph among different processing units because of the multiplicity of connections. Arbitrarily distributing the data structures in ways that span or cross multiple partitions will lead to increased cross-partition network traffic, which effectively eliminates the perceived performance benefit of data distribution.

10.9 FEATURES OF A GRAPH ANALYTICS PLATFORM

If your big data challenge is suited to a graph analytics solution, it is worth enumerating some key features to look for, both from the software and from the hardware platform perspectives. These can effectively be grouped into three sets of features: ease of development and implementation, interoperability with complementary reporting and analysis technologies, and system execution performance.

The ease of development features of the platform is enabled through the adoption of industry standards as well as supporting general requirements for big data applications, such as:

- **Seamless data intake:** Providing a seamless capability to easily absorb and fuse data from a variety of different sources.
- **Data integration:** A semantics-based approach is necessary for graph analytics to integrate different sets of data that do not have predetermined structure. Similar to other NoSQL approaches (as discussed in Chapter 9), the schema-less approach of the semantic model must provide the flexibility not provided by relational database models.
- **Inferencing:** The application platform should provide methods for inferencing and deduction of new information and insights derived from the embedded relationships and connectivity.
- **Standards-based representation:** Any graph analytics platform must employ the resource description framework standard (the RDF, see <http://www.w3.org/RDF/>) to use triples for representing the graph. Representing the graph using RDF allows for the use of the SPARQL query language standard (see <http://www.w3.org/TR/rdf-sparql-query/>) for executing queries against a triple-based data management environment.

Interoperability is critical, and here are some considerations for the operational aspects of the platform:

- **Workflow integration:** Providing a graph analytics platform that is segregated from the existing reporting and analytics environments will have limited value when there are gaps in incorporating results from across the different environments. Make sure that the graph analytics platform is aligned with the analysis and decision-making workflows for the associated business processes.
- **Visualization:** Presenting discoveries using visualization tools is critical to highlight their value. Look for platforms that have tight integration with visualization services.
- **“Complementariness”:** A graph analytics platform augments an organization’s analytics capability and is not intended to be a replacement. Any graph analytics capabilities must complement existing data warehouses, data marts, OLAP engines, and Hadoop analytic environments.

Finally, although all of the following are reasonable expectations for any big data platform, these criteria are particularly important for graph analytics due to the nature of the representation and the types of discovery analyses performed:

- **High-speed I/O:** Meeting this expectation for real-time integration requires a scalable infrastructure, particularly with respect to high-speed I/O channels, which will speed the intake of multiple data streams and thereby support graphs that are rapidly changing as new data is absorbed.
- **High-bandwidth network:** As a significant burden of data access may cross node barriers, employing a high-speed/high-bandwidth network interconnect will also help reduce data latency delays, especially for chasing pointers across the graph.
- **Multithreading:** Fine-grained multithreading allows the simultaneous exploration of different paths and efficiently creating, managing, and allocating threads to available nodes on a massively parallel processing architecture can alleviate the challenge of predictability.
- **Large memory:** A very large memory shared across multiple processors reduces the need to partition the graph across independent environments. This helps to reduce the performance impacts associated with graph partitioning. Large memories are useful in keeping a large part, if not all of the graph resident in-memory. By migrating allocated tasks to where the data is resident in memory reduces the impact of data access latency delays.

10.10 CONSIDERATIONS: DEDICATED APPLIANCES FOR GRAPH ANALYTICS

There are different emerging methods of incorporating graph analytics into the enterprise. One class is purely a software approach, providing an ability to create, populate, and query graphs. This approach enables the necessary functionality and may provide the ease-of-implementation and deployment. Most, if not all, software implementations will use industry standards, such as RDF and SPARQL, and may even leverage complementary tools for inferencing and deduction. However, the performance of a software-only implementation is limited by its use of the available hardware, and even using commodity servers cannot necessarily enable it to natively exploit performance and optimization.

Another class is the use of a dedicated appliance for graph analytics. From an algorithmic perspective, this approach is equally capable as one that solely relies on software. However, from a performance perspective, there is no doubt that a dedicated platform will take advantage of high-performance I/O, high-bandwidth networking, in-memory computation, and native multithreading to provide the optimal performance for creating, growing, and analyzing graphs that are built from multiple high-volume data streams. Software approaches may be satisfactory for smaller graph analytics problems, but as data volumes and network complexity grow, the most effective means for return on investment may necessitate the transition to a dedicated graph analytics appliance.

10.11 THOUGHT EXERCISES

When considering graph analytics, here are some questions and exercises to ponder:

- What are the main criteria you would use to determine whether to use a structured database approach or a graph analytics approach to addressing a particular business challenge?
- Of the most valuable three big data opportunities within your organization, which are most suited to a graph analytics approach?
- Would a graph analytics platform replace any existing production business intelligence and reporting applications? If yes, why, and if not, why not?
- Describe a pilot application for graph analytics.

This page intentionally left blank